Technical implementation document

Physics informed learning and verified inverse design of functionally graded plates

TID version 1.0 | 7 November 2026 | Proposed implementation specification

Architecture and current state

The implementation has five layers: mechanics reference, dataset generation, surrogate learning, evaluation and constrained design. Maintain a direct-solver path throughout so every learned prediction can be checked. The supplied archive implements only the first layer for a restricted simply supported family, plus tests and a demonstration.

The existing files are fgplate/material.py, fgplate/navier.py, validation/validate.py, validation/ritz_crosscheck.py and demo.py. Preserve these as the baseline. The supplied 23-check suite was rerun successfully. A stringent quadrature roundoff warning occurred in the Ritz routine; store it with the test log rather than suppressing it. Proposed modules below are not yet implemented.

Mechanical model and conventions

Use SI units internally: length in metres, modulus and pressure in pascals, density in kg/m3, frequency omega in rad/s, and displacement in metres. Report frequency in Hz only after dividing omega by 2 pi. Define the plate domain as 0 <= x <= a, 0 <= y <= b, -h/2 <= z <= h/2. Let g denote the gradation index to avoid confusion with the Navier mode index n.

The ceramic fraction is Vc(z) = (z/h + 1/2)^g. For g = 0 define Vc = 1 everywhere. For E and rho use P(z) = Pm + (Pc - Pm) Vc(z). Start with Ec = 380 GPa, Em = 70 GPa, rhoc = 3800 kg/m3, rhom = 2702 kg/m3, nu = 0.3 and ks = 5/6. These reproduce the handoff assumptions; they are not calibrated properties of a particular fabricated specimen.

Use first-order shear kinematics u = u0 + z phix, v = v0 + z phiy, w = w0. Define membrane strain e = [u0,x, v0,y, u0,y + v0,x], curvature kappa = [phix,x, phiy,y, phix,y + phiy,x], and transverse shear gamma = [phix + w,x, phiy + w,y]. Use consistent engineering-shear conventions in the constitutive matrices.

Compute A = integral Q dz, B = integral z Q dz and D = integral z^2 Q dz. The shear matrix is As times the 2 by 2 identity, where As = ks integral E/[2(1+nu)] dz. Compute mass moments Ij = integral rho z^j dz for j = 0, 1, 2. Keep B and I1 for asymmetric grading; setting either to zero without changing the reference surface changes the problem.

For constant nu, retain the exact closed-form thickness moments in material.py. Verify them independently against quadrature at fractional g and near the homogeneous limit. The finite-series solution is exact only within the stated plate theory and load truncation, subject to floating-point error.

Supports represented by the Navier basis

The basis is u0 = sum U cos(alpha x) sin(beta y), v0 = sum V sin(alpha x) cos(beta y), w0 = sum W sin(alpha x) sin(beta y), phix = sum X cos(alpha x) sin(beta y), and phiy = sum Y sin(alpha x) cos(beta y), with alpha = m pi/a and beta = n pi/b.

At x = 0,a this imposes v0 = w0 = phiy = 0 and is compatible with zero normal membrane force Nx and normal bending moment Mx. At y = 0,b it imposes u0 = w0 = phix = 0 and is compatible with Ny = My = 0. These are the specific mixed essential/natural conditions to reproduce in independent comparisons.

“Simply supported” alone is insufficient, especially when B is nonzero. Confirm the force and rotation conventions in the FE implementation before comparing results.

For a coordinate-based energy network, use u0 = eta(1-eta) Nu, v0 = xi(1-xi) Nv, w0 = xi(1-xi)eta(1-eta) Nw, phix = eta(1-eta) Nx and phiy = xi(1-xi) Ny, with xi = x/a and eta = y/b. These factors enforce the essential part above. Natural conditions arise from the variational problem and must still be checked. A sine/cosine modal representation provides an alternative exact embedding. Do not set both rotations to zero on every simply supported edge; that would add unintended restraint.

For the optional fully clamped case, explicitly define u0 = v0 = w0 = phix = phiy = 0 on the boundary. This is a different family and requires a different basis or boundary factors, independent labels and validation.

Solver work before producing the dataset

- Validate positive finite geometry, moduli, densities and ks; require g >= 0, valid mode indices and a physically admissible isotropic Poisson ratio. Reject unsupported property names rather than silently selecting density.
- Replace generic eigvals(solve(M,K)) with a symmetric generalized eigenproblem, for example scipy.linalg.eigh(K,M), after checking symmetry and positive definiteness. Do not silently discard imaginary or negative eigenvalues.
- Retain eigenvectors. Identify flexural modes using transverse participation and track modes when parameters vary. The five eigenvalues returned for one (m,n) pair are not the first five global plate bending frequencies.
- For multiple global modes, scan enough half-wave pairs and merge the flexural branches. Check convergence of the smallest frequencies as the scan grows. Do not rely on a universal ordering assumption for every future material or geometry.
- Extend load input to modal coefficients Qmn and add evaluation of u0, v0, w0, phix and phiy. The current API exposes only w even though the modal coefficients contain all five fields.
- Implement the vectorized static formula for Wmn as an independent competitor within its valid constant-nu model. Confirm agreement with the full coupled system before using it for timing.
- Record complete citations and table locations for the handoff's published benchmark numbers. Label same-theory checks separately from comparisons with higher-order or 3D elasticity.

For uniform loading Qmn = 16 q0/(pi^2 m n) for odd m,n and zero otherwise. For the existing sinusoidal load only Q11 = q0. The default cutoff of 49 means 625 active odd-odd modes, not 49 total modes. Establish convergence on centre deflection, field norms and any derivative quantity separately; convergence of one scalar does not establish stress convergence.

Dataset design and split protocol

Start with 200 pilot designs and inspect trends, range coverage and singular values. Then use a fixed nested pool with 200, 500, 1000 and 2000 labelled training designs for learning curves, plus 300 validation and 500 untouched in-domain test designs. Keep an optional 5000-design training expansion only if the pilot shows a need. These counts are plans, not generated data.

Use transformed Latin hypercube sampling over g, slenderness and aspect ratio. The primary interpolation domain is g in [0,8], a/h in [5,100] and b/a in [0.5,2], with a = 1 m as a computational scale. Sample log(1+g), log(a/h) and b/a, then invert the transformations. Include separate deterministic homogeneous and range-corner verification cases. For the g extrapolation experiment, withhold g in (8,10] completely from training, validation, POD construction and physics collocation.

Initially use the single sine and uniform loads for verification. For a richer field task, set q(x,y) = qscale sum c_mn sin(m pi xi) sin(n pi eta) over 1 <= m,n <= 5. Draw smooth coefficient mixtures, for example c_mn = z_mn/(m^2+n^2), normalize their Euclidean norm to one and store the resulting normalization. This defines a controlled signed load class; it is not automatically a one-sided physical pressure. Use nonnegative pressure families for a later physical loading application and compute their coefficients consistently.

Train on mixture realizations and test on unseen realizations. Add a held-out smooth load family, such as a normalized Gaussian patch projected into the same 5 by 5 basis. Separate the projection error of the actual patch from surrogate error on its truncated representation. Higher-frequency loads beyond this basis require basis enrichment; failure to represent them is not solved by fine-tuning branch coefficients.

Create a permanent design_id from geometry and material parameters. Keep all grid points, amplitudes and repeated load variants of a design in one partition for the main experiment. Freeze partitions before fitting scalers, POD modes or hyperparameters. If a supplementary study reuses geometries to isolate load-only generalization, report it separately. Physics collocation at a held-out parameter makes that case part of training and invalidates a claim of untouched generalization.

Store scalar vibration labels once per design because the present linear free-vibration problem does not depend on the static transverse load amplitude or shape. The bending response is linear in load amplitude; train on a normalized load and restore amplitude analytically. Do not create thousands of amplitude-scaled copies and count them as independent designs.

Data schema and normalization

Field group	Required contents
Identity	design_id, load_id, split, seed, generator version
Physics	a, b, h, g, Ec, Em, rhoc, rhom, nu, ks, support identifier
Loading	qscale, mode-index list, modal coefficients, load-family identifier
Labels	Five displacement/rotation fields or modal coefficients; frequency and mode identifiers
Numerics	Series cutoff, grid, convergence indicators, solver version, units
Provenance	Run time, environment versions, checksums, failure status

Store numeric arrays in NPZ or HDF5 and metadata in JSON; use CSV only for scalar summaries. Use a fixed 41 by 41 normalized evaluation grid for initial field reporting, with additional independent quadrature points for physics and error checks. Store modal coefficients so fields can be reconstructed at new points without interpolation ambiguity.

Use characteristic scales chosen before training: length a, displacement h and rotations h/a, followed by componentwise training-set scaling if necessary. The different y derivative scale is 1/b. Under the coordinate mapping d/dx = (1/a)d/dxi, d/dy = (1/b)d/deta and area integration carries ab. Include these factors in the energy and residuals. Do not confuse normalizing a coordinate with leaving its physical derivative unchanged.

Retain the handoff benchmark displacement wbar = 10 Ec h^3 w/(q0 a^4) for matching its tables. For multi-mode loads replace q0 with the explicitly stored positive load scale. Use omegabar = omega a^2/h

sqrt(rhoc/Ec) consistently in ML reporting. The handoff also uses omega h sqrt(rhoc/Ec); convert explicitly rather than mixing these two measures.

Model ladder and baselines

Model	Role	Required comparison
Original and optimized Navier	Numerical reference and timing competitor	Same outputs and tolerances
Polynomial or RBF response surface	Low-complexity scalar baseline	Same labels and splits
MLP and random forest	Scalar deflection and frequency regression	Tuned on validation only
GPR on a manageable subset	Small-data scalar baseline	Report sample and compute limits
POD coefficient regression	Reduced field baseline	Separate basis and regression error
Data-only reduced neural model	Architecture-matched baseline	Same basis and network size
Physics-informed reduced model	Main learning experiment	Change loss, not test set
Single-case energy PINN	Physics-loss verification	Include training cost per case

A fixed POD trunk plus a learned coefficient map is a reduced-basis surrogate, often described as POD-DeepONet. Name its structure precisely. A network taking only three scalar parameters is not evidence of learning arbitrary input functions. The varied-load experiment supplies a finite-dimensional load-function representation, and conclusions remain limited to that representation.

Build POD only from training fields. Scale the five components before concatenation so displacement units do not dominate rotations. Select rank from a training singular spectrum, then verify field reconstruction error on validation data. A 99.9 percent snapshot-energy criterion is a starting point, not a guarantee of small pointwise, derivative or physical-energy error. Use the smallest rank meeting the predeclared validation tolerance.

Represent each POD mode as a linear combination of the original smooth Navier basis functions when possible. Then evaluate derivatives analytically. A grid of stored POD values is not automatically differentiable in space; for FE-based modes use consistent FE interpolation and derivatives or a verified discrete energy formulation. Do not apply automatic differentiation to detached arrays and assume it yields spatial physics.

Physical objectives and training

For linear static bending, minimize the total potential energy

Pi = 0.5 integral [e^T A e + 2 e^T B kappa + kappa^T D kappa + gamma^T S gamma] dA - integral q w dA.

Here S is the 2 by 2 diagonal matrix with As on both diagonal entries. Use all five kinematic fields or a correctly derived condensed formulation. A loss based on w alone cannot simply omit the FG membrane-bending coupling.

For a modal model, the equivalent per-mode potential is proportional to 0.5 c^T K c - f^T c with the common area factor ab/4. This gives an inexpensive implementation check. Energy is signed and need not approach zero. Do not square total potential energy as the training objective. If using residual learning instead, normalize each equilibrium equation by an appropriate force or moment scale so units do not dominate the loss.

Proposed training: pretrain on scaled field or coefficient MSE; fine-tune with scaled potential energy plus the supervised term. Average the per-design loss after consistent physical scaling. Also test residual-based fine-tuning as a separately specified variant if needed. Compare at matched label counts and report the extra physical-evaluation cost. Preselect loss-weight candidates and choose on validation; avoid tuning against final test results.

Begin with a branch MLP of three hidden layers of 64 tanh units, batch size 32 and Adam learning rate 0.001 as a pilot configuration. These are starting settings, not optimized values. Use validation-based stopping and record maximum epochs and total wall time. Add L-BFGS only on a fixed deterministic quadrature set after the basic physics implementation works. Repeat the final comparison for seeds 11, 22 and 33.

Integrate energy with weighted tensor-product Gauss rules or a correctly weighted sampling scheme. Compare increasing orders, for example 16, 24 and 32, and refine further if necessary. If points are concentrated near boundaries, an unweighted mean generally does not approximate the uniform-area integral. Use appropriate quadrature weights or importance correction. Evaluate residuals and energies on a separate denser rule to detect quadrature exploitation.

For vibration, first use a separate scalar head trained on the reference eigenfrequencies. If a physics-informed eigenfield extension is added, minimize the Rayleigh quotient with mass normalization d^T M d = 1, so a zero field is inadmissible. Enforce mass orthogonality for higher modes and track signs/mode identities. Learning bending alone does not establish a physics-informed frequency predictor. Buckling needs geometric stiffness and a specified prestress state and remains out of scope.

Experiment matrix and validation gates

Run E0 for reference mechanics and convergence; E1 for scalar baselines; E2 for field rank and projection error; E3 for data-only versus physics-informed models across the nested label budgets; E4 for held-out gradation and load-family tests; E5 for verified inverse design. Run E6 for the clamped family only if its solver has passed verification.

Report relative field L2 error, normalized maximum absolute error, maximum deflection error, first-frequency error, boundary violations and normalized equilibrium residuals. Use max absolute deflection over the domain for general asymmetric loads; centre deflection is not necessarily the maximum. State whether grid maxima have been refined. Near-zero reference fields need a declared denominator floor or an absolute metric.

Proposed in-domain targets are median field error <= 1 percent, 95th percentile <= 3 percent; frequency median <= 1 percent, 95th percentile <= 2 percent. Report worst cases and parameter-resolved errors. R squared alone is insufficient and can be negative. Use paired case-level comparisons and report variation across training seeds; do not treat correlated grid points as independent samples.

Separate four errors: reference discretization/truncation, POD projection, learned mapping and plate-model discrepancy. Test complete fields and natural boundary terms, not only central values.

Inspect thin and thick limits, fractional g, extreme aspect ratios, load superposition, positive stiffness/mass and dimensional scaling. Use finite differences to check design gradients independently when gradient-based optimization is used.

For independent FEM verification, begin with about 20-30 designs covering the domain rather than only random central cases. Match material constants, supports, shear convention and geometry. Refine in-plane mesh and thickness discretization of the material grading independently. Use locking-resistant elements for thin plates. Compare to a same-theory FE formulation first; compare selected thick cases with higher-order or 3D models as a separate model-form exercise. If FEM is unavailable, report the study as verified within the plate theory and do not claim independent physical validation.

Inverse design specification

Use a fixed footprint, an explicitly declared service pressure q(x,y), and design variables theta = [h,g]. Keep h within the validated slenderness range. A simple engineering formulation is to minimize mass m = ab h [rhom + (rhoc-rhom)/(g+1)] subject to max|w| <= wlim, f1 >= fmin, and specified bounds on h and g. Add a ceramic-use limit h/(g+1) <= Cmax per unit area if it is part of the design requirement. Do not invent a manufacturing constraint without a reason.

Choose service limits after a direct-solver feasibility scan so the constraints are meaningful and not automatically satisfied or impossible. Report the baseline, units and feasibility envelope. Limit deformation to the agreed linear regime; w/h <= 0.1 may be used as a conservative screening rule, but it is not a universal proof of linear validity. Reject or flag designs needing nonlinear analysis.

Compare direct-solver optimization with surrogate-assisted optimization using the same bounds and stopping tolerances. Since there are only two variables, include a dense direct grid plus local refinement as a transparent competitor. Use multiple starts if needed. Reanalyse every final candidate and nearby perturbations with the reference solver; for the extended model also perform independent FE checks. Count corrective samples and retraining in the total cost.

Do not promise a unique inverse solution: different thickness-gradation pairs can meet the same target. Use a constrained optimization or Pareto formulation rather than assuming a single-valued response-to-design neural map. Report all constraint violations, solver-verified objective values and any optimality gap that can actually be established.

Runtime and reproducibility

Benchmark single-case latency and batch throughput separately. Include preprocessing, basis reconstruction, host/device transfers and output extraction. Warm up libraries, synchronize GPU timers and report repeated measurements with hardware, precision and thread count. Compare the same scalar or full-field output at the same accuracy. Never compare a network amplitude query with a solver's complete field and call it a like-for-like speedup.

Define offline cost as data generation plus training plus tuning, and incremental online time per query. A break-even estimate is Nbreak = Toffline/(Treference - Tsurrogate) only when the denominator is positive. Include reference precomputation fairly. A network slower than the optimized analytical solver has no timing break-even for that workload, although it may still be useful as a learning-method study.

Proposed directories: fgplate for mechanics; datasets for generators and split manifests; models for scalar and field predictors; physics for scaling, boundary conditions and energy; experiments for fixed configurations; design for constrained optimization; validation for mechanics and ML checks; reports for figures and tables. Each run records code version, environment, seeds, transforms, support definition, data hash, hyperparameters, loss terms, checkpoint selection and hardware.

Keep paper PDFs and private source material outside any public release unless redistribution is authorized. The project dataset should consist of the project's own simulation outputs, with cited equations and provenance. Preserve failed runs and negative findings in the experiment log.

First implementation milestone

The next concrete milestone is a reproducible 200-case pilot with validated modal loads, all five fields, fixed grouped splits, a snapshot-rank plot and a scalar baseline. Complete solver hardening and benchmark provenance first. Do not spend the initial milestone implementing every architecture or generating a large unexamined dataset.

Reference basis

The companion research review supplies full bibliography and page locators. Mechanics: supplied material.py and navier.py, plus Jin et al. [1] and Zhang et al. [16]. Learning: Zhang et al. [5], Shen and Zhao [9], Yang et al. [10], Kim et al. [11] and Ly et al. [15]. External prior art [A1]-[A4] constrains the novelty claim. Formulas and engineering choices proposed here must be verified in implementation; no cited paper independently validates this entire new pipeline.